

CDS View 課程練習 2：欄位選取與運算

Lecture

這一課要教什麼

cds01 建的 **ZI_CDS01_CARRIER** 欄位清單跟底層表 **SCARR** 幾乎一模一樣，只是重新列了一次——那一課的思考題其實已經點破：這樣包一層，本身沒有立即的好處。這一課 (cds02) 開始把 CDS View 的價值疊上去：**欄位不是只能照抄，可以改名、可以做運算、可以套用內建函數、可以依條件分類、可以加一個查詢本身沒有的常數欄位**。這些能力都是直接查表 (**SELECT * FROM sflight**) 做不到、或者要靠呼叫端程式額外寫邏輯才能做到的事，現在搬進 CDS View 裡統一維護。

這一課用標準表 **SFLIGHT** (訂位航班明細，含日期、價格、座位數) 當素材，建立 **ZI_CDS02_FLIGHT**。

語法元素逐一講解

在看完整範例之前，先分段認識這一課會用到的六種語法元素。

① **別名 (as)**：CDS View 的欄位清單可以把底層欄位改名成更有語意、或更貼近消費端習慣的名稱：

```
carrid as AirlineID
```

② **算術運算**：CDS 支援 **+**、**-**、*****、**/** 四則運算，可以直接對欄位做計算：

```
seatsocc / seatsmax
```

⚠ 但這裡有一個這系統實測出來的真實限制，非常值得注意：**/** (除法) 只允許用在浮點數 (**float**) 型別上，**SFLIGHT-SEATSOCC / SFLIGHT-SEATSMAX** 底層是整數型別 (**S_SEATSOCC/S_SEATSMAX**，NUMC/INT2)，直接寫 **seatsocc / seatsmax** 啟用會報錯：

```
Division x/y is only allowed for float numbers
```

先前試過把兩邊都 **CAST** 成 **abap.dec(5, 2)** (十進位小數) 依然不行，一樣報同樣的錯——**要轉成 abap.decfloat34 (十進位浮點數)** 除法才會過：

```
cast( seatsocc as abap.decfloat34 ) / cast( seatsmax as abap.decfloat34 ) * 100
```

③ **CAST**：把一個欄位或運算結果明確轉型成另一種型別，語法是 **cast(<來源> as <目標型別>)**。上面除法運算已經示範過一次；另一個常見用途是把日期型別 (**abap.dats**) 轉成字元型別，才能套用字串函數 (見下一點)。

④ **字串 / 日期內建函數**：

函數	用途	這一課用到的寫法
<code>concat(a, b)</code>	字串串接	<code>concat(carrid, connid)</code> ——把航空公司代碼 + 航班編號接成一組「航班號碼」
<code>substring(a, offset, length)</code>	取子字串	<code>substring(cast(fldate as abap.char(8)), 1, 4)</code> —— <code>fldate</code> 是 <code>abap.dats</code> 型別 (YYYYMMDD 八碼) · 要先 <code>CAST</code> 成 <code>abap.char(8)</code> 才能用字串函數在上面取年份
<code>dates_days_between(date1, date2)</code>	計算兩個日期相差幾天 (<code>date2 - date1</code>)	<code>dates_days_between(cast('20260101' as abap.dats), fldate)</code> ——算這個航班日期距離固定參考日 2026/01/01 差幾天 (負數代表更早)

⑤ **CASE WHEN**：依條件把一個欄位分類成不同的標籤值：

```
case
  when <條件1> then '值1'
  when <條件2> then '值2'
  else           '預設值'
end as 別名
```

⚠ ⚠ 這裡有這一課最重要的一個真實限制，寫程式前務必先知道：這系統的 CDS 編譯器 (V1 舊式 `define view`) 的 **CASE WHEN** 判斷條件只能寫純欄位 / 常數的比較，不能用運算式，也不能呼叫函數。這是實測一路踩出來的，過程中依序看到三種不同的錯誤訊息：

```
" 嘗試 1：條件裡直接寫乘法運算 (seatsocc * 100) >= (seatsmax * 80)
" → Unexpected word "*"

" 嘗試 2：條件裡改寫除法運算 (先 CAST 成 decfloat34)
" → Unexpected word "/"

" 嘗試 3：改用內建函數 division( seatsocc, seatsmax, 2 ) >= '0.80'
" → User-defined functions are not supported in the SEARCHED CASE WHEN clause

" 嘗試 4：改成引用同一個 SELECT 清單裡另一個已經算好的別名 OccupancyRatePercent
" → The column OccupancyRatePercent is unknown
```

四種嘗試分別代表：條件裡不能有運算子 (`*` / `/`)、不能呼叫函數 (連 CDS 內建函數 `division(...)` 都不行)、也不能引用同一句 `SELECT` 清單裡其他欄位的別名 (這點其實是 SQL / CDS 通用的規則，不是這系統特有——欄位別名要等整個 `SELECT` 清單都算完才「存在」，同一清單裡的欄位彼此不能互相引用)。這代表如果你想依「計算結果」做分類判斷，這一層 **View** 做不到——要嘛把判斷邏輯改成只用純欄位比較，要嘛把計算結果放進下一層 **View** 才能在那一層的 **CASE WHEN** 引用它 (後者正是 cds05「分層設計」要教的技巧，這裡先預告一下)。

這一課乖乖遵守這個限制，`OccupancyStatus` 的判斷邏輯改成只用欄位跟欄位、欄位跟常數的直接比較：

```

case
  when seatsocc >= seatsmax then 'FULL'
  when seatsocc <= 10       then 'MOSTLY_EMPTY'
  else                      'AVAILABLE'
end as OccupancyStatus

```

⑥ **常數欄位**：CDS View 可以在欄位清單裡直接放一個字面值常數，讓每一筆查詢結果都額外帶著這個固定值——常見用途是標記資料來源、版本號、或是給消費端一個「這筆資料是從哪個 View 查出來的」的識別欄位：

```
'CDS02' as DataSource
```

完整範例：**ZI_CDS02_FLIGHT**

```

@AbapCatalog.sqlViewName: 'ZICDS02FLGT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS02: Flight with Computed Fields'
define view ZI_CDS02_FLIGHT
  as select from sflight
{
  key carrid                                as AirlineID,
  key connid                                as ConnectionID,
  key fldate                                as FlightDate,

  concat( carrid, connid )                  as FlightNumber,

  dats_days_between( cast( '20260101' as abap.dats ), fldate ) as
DaysSinceYearStart,

  substring( cast( fldate as abap.char( 8 ) ), 1, 4 ) as FlightYear,

  cast( seatsocc as abap.decfloat34 )
    / cast( seatsmax as abap.decfloat34 )
    * 100                                    as OccupancyRatePercent,

  seatsmax                                  as SeatsMax,
  seatsocc                                  as SeatsOccupied,

  case
    when seatsocc >= seatsmax then 'FULL'
    when seatsocc <= 10       then 'MOSTLY_EMPTY'
    else                      'AVAILABLE'
  end                                       as OccupancyStatus,

  price                                    as Price,

```

```
currency                                as CurrencyCode,  
  
'CDS02'                                as DataSource  
}
```

跟 cds01 一樣，`sqlViewName` / `authorizationCheck` / `EndUserText.label` 三個基本 annotation 照舊，沒有新增別的 annotation——這一課的重點全部在欄位清單本身的表達能力。

Eclipse ADT 建立 CDS View：Step by Step

1. 對著 `$TMP` 套件右鍵 → **New** → **Other ABAP Repository Object** → 篩選 `Data Definition` → **Next**
2. Name：`ZI_CDS02_FLIGHT`，Description：`CDS02 Flight with Computed Fields`，Package：`$TMP` → **Next** (Transport 畫面直接 **Finish**)
3. Templates 畫面選 **Define View** (**obsolete as of AS ABAP 7.57**) (跟 cds01 一樣，不要選任何帶 `Entity` 字樣的模板)
4. Reference Object 選 `SFLIGHT` 帶出初始欄位清單
5. 改成上面「完整範例」的內容
6. **Ctrl+S** 存檔 → **Activate** (`Ctrl+F3`)
7. 右鍵 → **Open With** → **Data Preview**，確認查得到資料，`OccupancyRatePercent` / `OccupancyStatus` / `FlightNumber` 這些計算欄位都有值

這一課學到的東西，接下來會怎麼用

- cds03：Association——這一課的欄位運算都是「在單一表的欄位上做文章」，cds03 開始會把多張表的資料接起來
- cds05：分層設計——這一課遇到的「CASE WHEN 不能引用同一層算好的別名」，就是分層設計要解決的問題之一：把計算結果放進 Interface View，讓 Composite View 在下一層可以直接拿它當普通欄位使用 (含拿去做 `CASE WHEN` 判斷)
- cds07：聚合——`SUM`/`AVG`/`COUNT` 這類聚合函數用法跟這一課的內建函數概念相通，但有額外的 `GROUP BY` 規則

Eclipse ADT Step by Step (重點回顧)

1. 對著 `$TMP` 套件右鍵 → **New** → **Other ABAP Repository Object** → `Data Definition`
2. 填 Name / Description / Package，Transport 畫面直接 Finish
3. Templates 畫面選「Define View (obsolete as of AS ABAP 7.57)」
4. 選 `SFLIGHT` 當 Reference Object
5. 改成本課要求的完整內容
6. `Ctrl+S` 存檔 → **Activate**
7. 右鍵 → **Open With** → **Data Preview**，確認計算欄位都有值

學習目標

- 能用 `as` 幫 CDS View 欄位取一個跟底層技術欄位名不同的語意化別名
- 能寫出四則運算，並知道「除法只允許浮點數 (`abap.decfloat34`)，整數/定點小數欄位要先 `CAST` 才能相除」這個這系統實測出來的具體限制

- 能用 **CAST** 明確轉換欄位型別，知道字串函數（如 **SUBSTRING**）要作用在日期型別欄位上時，必須先把日期 **CAST** 成字元型別
- 能寫出至少一個字串函數（**CONCAT** / **SUBSTRING**）跟一個日期函數（**DATS_DAYS_BETWEEN**）
- 能寫出 **CASE WHEN ... THEN ... ELSE ... END** 多分支判斷，並且知道這系統的 **CASE WHEN** 判斷條件只能是純欄位 / 常數比較，不能寫運算式、不能呼叫函數（含 **CDS** 內建函數）、也不能引用同一句 **SELECT** 清單裡其他欄位的別名——能講出這三種限制各自對應的錯誤訊息
- 知道當真的需要「依計算結果分類」時，正確的做法是把計算結果放進下一層 View 才能引用（**cds05** 分層設計的預告）
- 能在欄位清單裡加一個常數欄位（字面值直接當作某個欄位的值）

物件清單

物件	名稱	型別
CDS Interface View	ZI_CDS02_FLIGHT	DDL/DF
驗證程式	ZR_CDS02_DEMO	PROG/P

全部物件都在 **\$TMP** 套件，已建立並啟用（讀取 **active** 版本內容與寫入內容逐字相符，代表沒有殘留未啟用版本）。

動手練習物件（你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡）：

物件	建議名稱	型別	對應練習
CDS View（基於 SPFLI ）	ZI_CDS02_FLIGHT_TIME （自訂）	DDL/DF	動手練習

動手練習

輪到你了：用標準表 **SPFLI**（航班基本資料，欄位含 **carrid** / **connid** / **cityfrom** / **cityto** / **deptime** / **arrtime** / **fltime**）建一個全新的 CDS View（物件名稱、套件 **\$TMP**，命名自訂，例如 **ZI_CDS02_FLIGHT_TIME**），要求至少包含：

1. 至少一個別名：把 **cityfrom** / **cityto** 改名成更語意化的名稱（例如 **DepartureCity** / **ArrivalCity**）
2. 至少一個算術運算：**fltime** 是飛行時間（分鐘），算一個「飛行時數」欄位（**fltime** 除以 60）——這一題故意讓你親自踩一次「除法要用浮點數」這個限制，先直接寫 **fltime / 60** 試著啟用，看看會不會報跟講義裡一樣的錯誤，再改成 **cast(fltime as abap.decfloat34) / 60** 修正
3. 至少一個字串或日期函數：例如用 **CONCAT** 把出發城市跟抵達城市接成一個「航線」欄位（**concat_with_space(cityfrom, cityto, 1)**，中間可以查一下這個函數的第三個參數是幾個空白字元）
4. 一個 **CASE WHEN**：依 **fltime** 分類成「**SHORT_HAUL**」（例如 **fltime <= 180**，180 分鐘以內） / 「**LONG_HAUL**」（其他）——只能用純欄位跟常數比較，不要嘗試在條件裡放運算式，親自驗證一次講義提到的限制
5. 一個常數欄位：標記這是你自己建的練習物件，例如 **'MY_EXERCISE' as Source**

建好、啟用成功後跟我說一聲（貼程式碼或截圖都可以），我會幫你核對語法有沒有踩到這系統的已知限制。

如果你想額外挑戰一下：試著故意在 **CASE WHEN** 條件裡寫一個運算式（例如 **when fltime * 2 > 300 then ...**），實際看一次啟用失敗的錯誤訊息，再改回純欄位比較——親眼看過這個錯誤，比只是讀講義印象更

深。

驗證方式

CDS View 本身不能直接執行，這一課用 `ZR_CDS02_DEMO` 透過 `programrun` 無頭驗證，同一批航班資料分別用 Open SQL 直查 `SFLIGHT` 跟查 `ZI_CDS02_FLIGHT`，確認：

1. 筆數一致
2. `FlightNumber` 正確把 `carrid + connid` 接成一組（例如 `AA + 0017 → AA0017`）
3. `OccupancyRatePercent` 計算正確（`seatsocc / seatsmax * 100`，例如 `372/385 → 96.62%`）
4. `OccupancyStatus` 正確依 `seatsocc/seatsmax` 分類（測試資料都遠低於滿座，全部落在 `AVAILABLE`）
5. `DataSource` 常數欄位每一筆都固定是 `'CDS02'`

實際執行結果（`programrun` 無頭執行）：

```
=== 1. Direct table SFLIGHT (Open SQL) ===
AA  0017  2018/10/29      372  /      385
AA  0017  2018/11/30      374  /      385
AA  0017  2019/01/01      370  /      385
AA  0017  2019/02/02      373  /      385
AA  0017  2019/03/06      373  /      385
=== 2. CDS View ZI_CDS02_FLIGHT (computed fields) ===
AA0017  2018      2621-  96.62337662337662337662337662337662  AVAILABLE  CDS02
AA0017  2018      2589-  97.14285714285714285714285714285714  AVAILABLE  CDS02
AA0017  2019      2557-  96.1038961038961038961038961038961  AVAILABLE  CDS02
AA0017  2019      2525-  96.88311688311688311688311688311688  AVAILABLE  CDS02
AA0017  2019      2493-  96.88311688311688311688311688311688  AVAILABLE  CDS02
=== 3. Row count match ===
MATCH: row counts equal          5
```

`FlightNumber`（`AA0017`）、`FlightYear`（`2018/2019`）、`OccupancyRatePercent`（`372/385×100 ≈ 96.62`）、`OccupancyStatus`（全部 `AVAILABLE`，因為測試資料座位都還沒滿）、`DataSource`（固定 `CDS02`）全部正確；`DaysSinceYearStart` 顯示負數（如 `2621-`，ABAP `WRITE` 負數的顯示慣例是把負號放在數字後面）也正確——因為這些測試航班日期都在參考日 `2026/01/01` 之前，距離參考日自然是負值。

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

1. 這一課發現「CASE WHEN 條件不能引用同一句 SELECT 清單裡的別名」，如果你真的需要依 `OccupancyRatePercent` 這個計算結果做分類，除了改成只用純欄位比較，還有什麼做法可以繞過這個限制？（提示：回顧「CASE WHEN」那一節的預告）
2. `DaysSinceYearStart` 用固定字串 `'20260101'` 當參考日寫死在 CDS View 裡，這樣設計有什麼潛在問題？如果想要參考日永遠是「今天」，你覺得該怎麼改？（這是 `cds04 Session Variables` 要教的內容，這裡先想想看）
3. 除法要求浮點數（`abap.decfloat34`）才能運算，但 `SeatsMax / SeatsOccupied` 這兩個欄位在 `ZI_CDS02_FLIGHT` 裡還是保留原本的整數型別直接輸出，沒有轉成浮點數——為什麼只有拿去做除法運

算的那一刻才需要 CAST，直接輸出整數欄位卻不需要？

4. 如果 `DataSource` 這個常數欄位的值以後可能會隨著環境（開發/測試/正式）不同而改變，你覺得寫死字面常數 `'CDS02'` 是好的設計嗎？有沒有更靈活的做法？（提示：這也是 `cds04 Session Variables` 的方向）

答案

見 `zi_cds02_flight.ddls.abap`、`zr_cds02_demo.prog.abap`。SAP 端物件：`ZI_CDS02_FLIGHT`（CDS View）、`ZR_CDS02_DEMO`（驗證程式）。動手練習（基於 `SPFLI` 的計算欄位 CDS View）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。